CameraReady: A Gated, Verification-First Workflow for Agentic Academic Paper Production

Yanwei Liu
Independent Researcher

***Abstract*—If you ask a large language model (LLM) agent to write a survey, it will happily do so, and it will invent citations while doing it. Most current defenses work after the fact: they scan a finished draft, guess which references look suspicious, and rely on network calls that fail when the API is rate-limited. This report describes CameraReady, an open-source authoring framework built on a different assumption: if a property can be checked mechanically, the workflow should check it, not the model. Concretely, the framework provides a ten-phase pipeline from research ideation to an arXiv-ready package, with fail-stop gates that refuse to advance until file-backed conditions hold, a citation pipeline in which references can enter `ref.bib` only by resolving against DBLP, CrossRef, or arXiv, and declarative configuration files for eight disciplines, six citation styles, and six conference venues. Where a check requires judgment, such as whether a cited paper actually supports the sentence citing it, the framework flags the case for the author instead of deciding silently. We verified the implementation with 545 unit tests across 34 modules and twelve injected-fault scenarios, all of which the blocking gates rejected. Writing this very paper with CameraReady doubled as an end-to-end case study: all 45 references originated from registry queries, and repeated re-verification under live network conditions surfaced transient throttling (38–45 verified entries per run), which illustrates why the tool persists bibliographic provenance locally rather than re-resolving remotely. A controlled comparison against unconstrained authoring agents remains future work. CameraReady is open source at https://github.com/e96031413/camera-ready.**

***Index Terms*—LLM agents, scientific writing, citation hallucination, workflow verification, research software**

I. Introduction

A citation makes a promise: the paper it points to exists, and it actually says what the citing text claims. Generative language models break both halves of that promise routinely. In an audit of 111 million citations across arXiv, bioRxiv, SSRN, and PubMed Central, non-existent references rose sharply alongside LLM adoption [1]. Fabricated citations have already appeared in published conference proceedings [2], and systematic evaluations find that fabrication persists across all major commercial and open-weight models [3].

Meanwhile, research agents have grown genuinely capable of writing papers end to end. The AI Scientist [4] and Agent Laboratory [5] automate ideation, experimentation, and paper compilation; later systems stretch agentic research over multi-day lifecycles [6], [7] and distributed teams [8], [9]. Structured generation pipelines enforce argument outlines before drafting [10], and surveys document rapid adoption across scientific domains [11]–[13]. What these frameworks largely do not have is a story about manuscript integrity: correctness is delegated to downstream benchmarks [14] or to post-hoc screeners that inspect the draft only after bad text exists [15], [16].

Post-hoc checking has a structural weakness: it sees the error after the error is written. Retrospective checkers lean on similarity thresholds and third-party web APIs, so they both miss fabrication and fail outright under rate limits and network outages. Cornelio et al. [17] argue that verification must instead live inside the research and authoring loop, and auditable protocols have been proposed for hypothesis formation [18], [19]. What has been missing is the same treatment for the authoring pipeline itself: fail-stop gates, deterministic citation grounding, and checkable repository conditions.

CameraReady fills that gap. It is an open-source workflow of 68 modular Python scripts (≈21.2k lines of code) that takes a manuscript from ideation to a pre-submission package passing local format and integrity gates. The operating principle is simple: a model's claim that a phase is done is not evidence; the phase ends when the repository says so. Within a cooperative agent model, where the toolchain governs repository state, verification splits into three tiers. *Enforced invariants* are mechanically decidable checks, such as whether a bibliography entry exists in a canonical registry or whether a reported number matches regenerated data, and failing one halts the workflow with a non-zero exit code. *Advisory diagnostics* handle what needs judgment, such as citation-to-abstract overlap or prose repetition, and report findings without blocking. *Human governance* keeps the decisions that should never be automated: novelty, claim adjudication, and the choice to submit.

Our primary contributions are fourfold: 1) *Three-tier gated workflow* (Section VI): a state machine governing ten phases through fail-stop invariant gates, advisory diagnostics, and an append-only override journal; 2) *Citation provenance by construction* (Section VII): a bibliographic pipeline that decouples parametric memory from `ref.bib`, admitting entries only via direct API resolution against DBLP, CrossRef, or arXiv; 3) *Provided declarative schemas* (Section V): configuration profiles for 8 disciplines, 6 citation styles, and 6 venues, so that supporting a new publishing standard means writing a configuration file, not editing verification code; and 4) *Artifact-*

This document is an engineering technical report accompanying CameraReady (release `v0.2.0`, commit `29a8fdb6a539f4b4e7cfb86988e6691714146e69`); it has not undergone formal peer review. Source code, evaluation records, citation provenance manifest, and reproduction scripts: https://github.com/e96031413/camera-ready.

level verification and provenance analysis (Section VIII): software verification across 545 unit tests in 34 modules, 12 negative-control fault injections, and an operational study of fetch-time provenance persistence under remote API rate limits; controlled empirical comparisons against unconstrained agent baselines remain future work.

II. Background and Related Work

A. Autonomous research and writing agents

LLM-based scientific agents have moved from helping with code to running research pipelines. Lu et al. [4] demonstrated fully automated research loops closed by automated peer review, and Agent Laboratory [5] divides the work among specialized agents for literature review, experimentation, and report writing. Long-horizon systems such as ScienceFlow [6], Kosmos [7], and FARS [8] scale this to distributed multi-agent collaboration, and goal-evolving architectures [9] adjust hypotheses as they go. On the manuscript side, Story2Proposal [10] enforces structural outlines before drafting, and surveys document rapid adoption across scientific domains [11]–[13]. Wang et al. [20] and Takahara et al. [18] argue for moving research validation out of the model and into auditable execution environments. CameraReady applies exactly this idea to paper writing: model self-evaluation is replaced by deterministic filesystem inspection.

B. Citation hallucination: measurement and detection

How often do models invent references? Often enough that it has its own literature. Zhao et al. [1] estimate that tens of thousands of fabricated citations have entered scholarly preprints, Naser [3] shows fabrication is widespread across commercial and open-weight models alike, and Chen et al. [21] trace the behavior to field-specific neurons in the parametric representations, suggesting that prompt-level controls alone have not eliminated citation fabrication across evaluated settings.

The standard response is detection after the fact: post-hoc checkers verify citations through retrieval-augmented search (CiteCheck [16]), internal attribution tracing (FACTUM [22]), existential queries (CheckIfExist [23]), and multi-agent cross-examination [24]. Rao et al. benchmark citation errors in publishing agents [25] and evaluate automated correction [15], and MCP servers support interactive bibliographic repair [26]. But grounding alone cannot eliminate invented citations [27]. These tools inspect drafts that already contain the error. CameraReady instead enforces bibliographic provenance before drafting begins, preventing one important class of identity-level fabricated references from entering the supported drafting path.

C. Verification and provenance in agent architectures

Verification is also creeping into agent architectures generally. CARE [28] statically checks shell commands before execution; TRACER [29] attaches cryptographic provenance to multimodal outputs; and She et al. [30] track provenance to detect policy drift. Souza et al. [31] formalize reference architectures for workflow provenance. For iterative loops, Wu et al. [32] derive bounded stopping rules and Sah et al. [33] model the "verifier tax" that trades safety against throughput. Tool evolution studies [34] and programmatic skill workflows [35] similarly trade free-form model autonomy for structured execution. CameraReady inherits this philosophy and applies it to one artifact in particular: the paper.

D. Review, detection, and reporting norms

Automating scientific review carries its own risks [36], [37], and machine-generated text now permeates both submissions [38] and reviewer comments [39]. Span classifiers attempt to detect synthetic prose [40], while reproducibility checklists [41] and automated code audits [42] offer more objective reporting criteria. Systematic review pipelines [43]–[45] impose strict reporting compliance for the same reason. CameraReady takes the verifiable subset of these norms, the structural, bibliographic, and procedural rules, and turns them into machine-readable schemas; judgment about scientific quality stays with the author.

III. Threat Model

The agent we have in mind is not malicious. It runs through ordinary shell and filesystem interfaces, tries to write a paper for a peer-reviewed venue, and fails in the ordinary ways that generative models fail. Four failure modes account for most of the damage:

F1: Fabricated or corrupted citations. The agent emits syntactically valid BibTeX for papers that do not exist, or attaches real authors to invented titles [25]. Autoregressive sampling simply cannot tell recall from fabrication.

F2: Claims unsupported by evidence. The manuscript reports numbers with no experimental artifact behind them, or cites papers for claims they never make.

F3: Formatting and venue policy violations. The document breaks page limits, omits required sections, ignores style guidelines, or leaks author identity in a double-blind submission.

F4: Premature workflow progression. The agent declares a phase finished because the conversation felt finished, then drafts on top of artifacts that do not exist.

CameraReady deterministically enforces the **mechanically checkable subset** of F1–F4: canonical bibliographic existence and registry resolution (part of F1), numerical consistency between reported tables and registered script outputs (part of F2), adherence to declarative venue formatting schemas (F3), and physical existence of phase prerequisite artifacts (F4). What it deliberately does not try to decide is higher-order semantic validity, such as whether a cited paper genuinely supports a nuanced conceptual claim, or whether an experimental methodology is sound. Those properties are handled by advisory diagnostics and, ultimately, by the author.

Operating model and trust boundary. We assume the agent cooperates with the CameraReady CLI and toolchain. Under that assumption, compliant execution cannot silently inject ungrounded citations or skip unfulfilled phase prerequisites, because the toolchain provides no code path for either. We do not model an adversarial agent deliberately tampering with

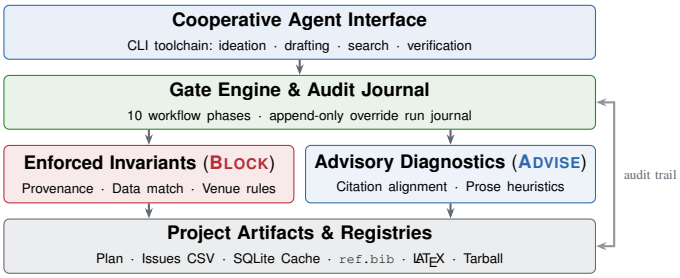

Fig. 1. System architecture and verification tiers: the gate engine enforces mechanically checkable invariants (BLOCK), reports advisory diagnostics (ADVISE), and preserves an audit trail over the physical project artifacts.

repository state through out-of-band shell commands; that threat requires OS-level sandboxing (discussed in Section IX).

IV. Design Principles

Five principles, each answering one of the failures above:

P1: Keep state in files, not in the conversation. Execution state lives in structured filesystem artifacts (JSON, Markdown) rather than ephemeral conversational memory, so a workflow can be paused mid-phase and resumed days later with nothing lost.

P2: Fail by default. When an invariant fails, the gate exits non-zero and says exactly which condition is unmet; 65 of 68 scripts behave this way (the remaining 3 are shared library modules with no independent entry point). Advancing requires passing every check or recording an explicit, justified override. Advisory diagnostics warn without blocking.

P3: No raw BibTeX from memory. Within the supported CLI there is simply no code path that writes model-generated BibTeX into `ref.bib`. Entries arrive only through API resolution against canonical registries, and unresolved queries emit explicit markers.

P4: Rules are data, not prompts. Field guidelines, citation conventions, and venue constraints are stored as declarative schemas, not as natural-language instructions that a model may reinterpret.

P5: Humans sign off where it matters. Three governance boundaries, approving the research plan, adjudicating extracted claims, and authorizing final submission, are enforced as explicit, file-backed sign-off checkpoints that the workflow itself will not skip.

V. System Architecture

CameraReady is organized into four architectural layers (Fig. 1): declarative rules, the gate engine, modular verification checkers, and physical project artifacts.

A. Rules as declarative data

Every discipline has its own idea of what a complete paper looks like, so CameraReady encodes that idea as a YAML profile rather than hoping a prompt captures it. A profile defines the mandatory sections, citation conventions, required statements (ethics, preregistration, data and code availability, conflicts, funding), the reporting guidelines the field recognizes (for example PRISMA for systematic reviews or CONSORT for randomized trials), and the pitfalls that commonly get papers in that field rejected. The computer science profile, for instance, mandates a data/code availability statement and flags unsupported performance claims and missing baselines. Venue configurations similarly capture page limits, column layouts, required sections, review models (single- or double-blind), and authoritative style templates. Table I summarizes the coverage shipped out of the box: 8 disciplines, 6 citation styles, and 6 venues. Because rules are data, adopting a new venue means writing one configuration file, not modifying verification code.

TABLE I
PROVIDED CONFIGURATION PROFILES AND SPECIFICATION SCHEMAS

Dimension	Count	Configured Profiles and Validation Status
Disciplines	8	CS (exercised); Medicine, Psych., Educ., Bus., Soc. Sci., Hum., Law (schema)
Styles	6	IEEE (exercised); APA 7, MLA 9, Vancouver, Chicago (schema)
Venues	6	NeurIPS, ICML, ICLR, ACL, AAAI, CVPR (CFP schema)
Formats	4	LaTeX (native); DOCX, Markdown, Typst (via Pandoc)

Fig. 2. The ten workflow phases (* indicates mandatory human-in-the-loop sign-off; red phases cannot close autonomously; non-linear re-entry is supported).

B. Execution engine and project artifacts

The core engine is a state machine over repository state. Its state file tracks the research topic, discipline, citation style, active phase, designated human approver, and logged overrides. A single command (`quick_start.py`) scaffolds a buildable skeleton with no prose in it, so a new project starts compilable from minute one. Fig. 2 shows the canonical milestone sequence from ideation to final packaging, but real authoring is never linear: writers cycle between drafting, literature refinement, and experiments. The workflow therefore supports phase re-entry, issue reopening, and iterative refinement across all phases, and refinement markers protect finalized sections by making modification scripts abort rather than overwrite polished work. Nor does the workflow end at the manuscript: the same gates drive rebuttal handling once reviews arrive, and post-acceptance outputs including Beamer slides, PowerPoint decks, HTML posters, and talk videos can be produced from the finished paper.

VI. The Gate Model

Fig. 2 shows the ten phases of the authoring workflow. What makes the model more than a diagram is that gate conditions are evaluated against physical repository files, not against anything the model says: `notes/research-question.md` must exist with non-empty content, the bibliography must meet the configured reference threshold without unresolved keys, and every task in the issues CSV must be complete with verified acceptance criteria.

TABLE II
SYSTEM GATE INVENTORY BY FUNCTIONAL CATEGORY

Category	Gate Name	Target Invariant / Diagnostic	Type
Workflow	autopilot	Phase exit predicate holds	BLOCK
	kickoff	Plan approved before issues	BLOCK
	rebuttal gate	Review feedback itemized	BLOCK
Citation	verify citations	3-layer lookup + review	BLOCK
	citation style	Required schema metadata	BLOCK
	BibTeX audit	Duplicates, years, recency	BLOCK
	citation align	Sentence-abstract lexical overlap	ADVISE
Integrity	data gate	Numbers match regenerated data	BLOCK
	integrity gate	Claims, privacy, bibliography	BLOCK
	claim registry	Assertion extraction worksheet	ADVISE
	prose diagnostic	Repetitive n-gram pattern density	ADVISE
Venue / Ops	format gate	Page limit, sections, macros	BLOCK
	anonymity	Author macros, URLs, emails	BLOCK
	privacy scan	Sensitive secrets/keys in repo	BLOCK
	portability	Cross-platform script sanity	BLOCK
	venue rules	Sync with published CFP guide	BLOCK

When the agent requests a phase transition, the gate script inspects the filesystem for itself. If a predicate fails, the gate exits with status code 1 and prints the specific unmet prerequisite, so the agent (and the author) knows exactly what is missing rather than receiving a vague refusal. Real research does produce legitimate exceptions, so the system supports a manual override: passing `--force` with a required justification advances the phase, but the justification becomes a permanent, append-only record in the run journal:

```
forced past 'ideate' despite:
notes/research-question.md is missing.

Reason: demonstration of override journaling
```

Because journal entries are append-only through the CamerAReady CLI state machine and cannot be purged via standard commands, every deviation stays visible forever. Table II summarizes the full gate inventory across all functional categories.

VII. VERIFICATION MECHANISMS

A. The citation path

Fig. 3 shows how bibliographic entries reach `ref.bib`, and just as importantly, how they cannot. The agent is free to formulate search queries, but the CLI toolchain offers no programmatic path for injecting raw BibTeX. Title queries go to DBLP and CrossRef, identifier queries to arXiv, and whatever comes back is normalized into a local SQLite registry before export. When a lookup fails, `fetch_bibtex.py` exits cleanly with status 0 but emits an explicit [VERIFY] marker and zero BibTeX records: the miss is recorded, not papered over. Enforcement happens downstream. Gates such as `verify_citations.py`, `integrity_gate.py`, and `arxiv_package.py` scan `ref.bib` and halt with exit code 1 the moment they find an unresolved [VERIFY] tag, so a draft containing an unverifiable reference cannot reach the packaging stage.

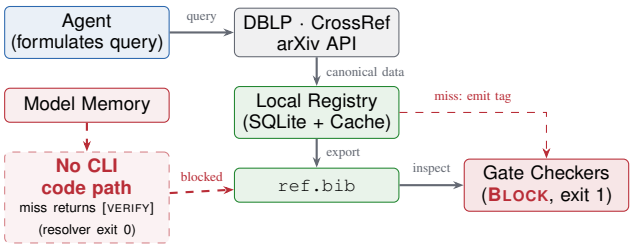

Fig. 3. Bibliographic dataflow and trust boundary: The dashed path from model memory to `ref.bib` is absent from the CLI toolchain. Query misses emit [VERIFY] tags (resolver exit 0), which are subsequently trapped by downstream blocking gates (exit 1).

B. Resilience to external service disruptions

Canonical registries are web services, and web services go down. Lookups therefore route through a persistent local cache indexed by query hash, so previously resolved citations survive API outages and rate limits. Fully network-isolated settings can point the tool at an offline JSON Lines corpus of pre-indexed papers via `--offline`. Cached records only ever corroborate citations that were verified when first fetched; a query with no cached or indexed answer yields an unverified marker, never a fabricated entry.

C. Reconciling empirical data and reported metrics

Numbers drift the same way citations do: a table is regenerated, the prose keeps yesterday's value, and nobody notices. The data gate closes this hole by binding each LaTeX table and figure to the data-generation script that produced it. When the gate runs, it re-executes the script, extracts the numerical tokens in the associated float, and checks that the reported numbers match the regenerated outputs within numerical tolerance, ignoring formatting numerals. A number the analysis does not produce fails the gate.

D. Semantic citation alignment

Some citation errors are subtler than fabrication: the cited paper exists but does not actually support the claim, making it a real reference acting as a false witness. The alignment checker addresses this by computing lexical and TF-IDF overlap between each citing sentence and its cited abstract. As an advisory diagnostic (ADVISE), it ranks the weakest pairings for the author to read rather than pretending to judge conceptual validity automatically.

E. Claim extraction, prose diagnostics, and packaging

Before packaging, the claim extractor parses the manuscript into a structured registry of factual assertions, and the integrity gate requires explicit human sign-off on every claim before the paper can ship. Prose diagnostics evaluate repetitive n-grams informatively, without blocking compilation. On the venue side, the format gate validates page limits and required sections, the anonymity checker flags author identifiers for double-blind submissions, and the packaging script bundles verified sources with inlined bibliographies into an arXiv-ready archive.

TABLE III
NEGATIVE CONTROL INTERCEPTIONS UNDER FAULT INJECTION

Injected Fault Condition	Gate Diagnostic / Interception	Exit
Ideate without RQ	Flags missing `research-question.md`	1
Kickoff without approval	Halts: plan approval checkbox unmarked	1
Forced override w/ reason	Advances phase, logs rationale to journal	0
Missing IEEE `journal`	Rejects entry: mandatory key missing	1
Signed paper anonymity	Flags 2 author macros, 1 URL (3 blockers)	1
NeurIPS format breach	Flags missing sections and style	1
Unadjudicated claims	Halts packaging: claim registry unapproved	1
Fabricated paper title	Emits [VERIFY]; 0 BibTeX (resolver 0; gate 1)	0
Mismatched table numbers	Traps discrepancy between prose and data	1
Failing analysis script	Catches non-zero pipeline exit code	1
Untracked rebuttal point	Halts: unmapped reviewer item	1
Stale venue page limits	Catches outdated page count constraint	1

TABLE IV
THREE-LAYER BIBLIOGRAPHIC VERIFICATION STABILITY ACROSS SIX REMOTE LOOKUP RUNS (SKIPPED DENOTES TRANSIENT REMOTE LOOKUP THROTTLING, NOT HALLUCINATION; UNION REFLECTS DISTINCT ENTRIES VERIFIED ACROSS RUNS).

Run	Verified	Suspicious	Hallucination Verdict	Skipped (Throttled / 429)
Run 1	39	0	0	6
Run 2	38	0	0	7
Run 3	41	0	0	4
Run 4	38	0	0	7
Run 5	45	0	0	0
Run 6	45	0	0	0
Union (1–6)	**45**	**0**	**0**	**0 (complete)**

VIII. SOFTWARE VERIFICATION AND SELF-APPLICATION

Evaluations ran on a Windows 10 workstation (build 10.0.19045, Python 3.13.9, MiKTeX); all reproduction commands and raw outputs are documented in `notes/experiment-results.md`. We assess CameraReady four ways: software verification of the implementation itself (545 unit tests across 34 modules), synthetic negative-control fault injections, an operational comparison of bibliographic provenance under live API conditions, and an end-to-end case study in which this very paper was written with the framework. Gate correctness is evaluated under injected faults; controlled comparison protocols against ungated agent baselines remain future work.

A. Unit verification and gate determinism

The test suite spans 545 unit tests across 34 modules and completes in 48.87 s with a 100% pass rate. Continuous integration validates every commit across a 3-OS (Ubuntu, macOS, Windows) $\times$ 2-Python-version matrix with CLI smoke tests. The portability checker verified zero OS-specific assumptions across 220 files in `--docs` mode (102 scripts/tests, 118 docs), and the specification validator reported zero configuration warnings. Of the framework's 68 scripts, 65 are standalone CLI entry points and all 65 implement fail-stop non-zero exit codes; the remaining 3 are library modules imported by other scripts. The worked example compiles in 5.8 s.

B. Gate enforcement under negative controls

We deliberately broke things on purpose to see whether the gates would notice. Twelve synthetic negative-control fixtures represent intentional violations of each gate's invariant (Table III); in every blocking case, the gate intercepted the non-compliant artifact and exited with status 1. Fed a fabricated title ("Gated Diffusion Transformers for Bibliographic Provenance Repair"), the resolver emitted an unresolved marker with zero BibTeX records rather than an entry. The data gate likewise intercepted an injected discrepancy (0.88 reported vs. 0.71 produced).

C. Heuristic checker refinements

The heuristics improved through use. Early claim extractors tokenized raw LaTeX markup into 48 noisy claims, one of which was a vector graphic; an AST cleaning pre-pass reduced this to 26 substantive assertions. Likewise, moving prose-logic checks to whole-document TF-IDF cosine similarity cut false-positive topical-disconnect warnings on this manuscript from 14 to 1.

D. Provenance by construction vs. post-hoc screening

In this case study, all 45 cited references were resolved via canonical arXiv queries and exported into `ref.bib`, with their persistent identifiers recorded in `citation-provenance.jsonl`. To test how that record holds up, we ran the three-layer verifier six times one afternoon under normal network conditions (Table IV). No identity-level fabricated bibliographic records were observed under the stated registry criterion, but verified entries fluctuated between 38 and 45 purely because of transient upstream throttling (recorded in `notes/experiment-results.md`). Tested with three fabricated citations lacking identifiers, the verifier returned skipped rather than hallucination verdicts. Declining to guess is the right call, but an inconclusive verdict provides no safety guarantee, and this is exactly the gap that persisting canonical provenance in SQLite at ingestion time closes: the workflow does not have to re-resolve remotely what it already proved locally.

E. Dogfooding self-application case study

Finally, we used CameraReady to write the paper you are reading. The manuscript went end to end through ideation, literature gathering, plan approval, issues tracking (38 to 45 tasks), experiments, and drafting. During review, the rebuttal gate intercepted five unaddressed comments and mapped them automatically to tracked issues via `--emit-issues`. One manual override was exercised during drafting, when 16 presentation tasks were skipped; the transition and its rationale were permanently recorded in the run journal, exactly as P2 and P5 promise.

IX. DISCUSSION, LIMITATIONS, AND FUTURE WORK

Validity, trust, and bounds. CameraReady enforces syntactic and bibliographic invariants; judging scientific novelty or conceptual validity remains the author's responsibility [14]. The framework assumes a cooperative agent, so isolation against untrusted shell agents requires OS-level sandboxing. Textual heuristics assist rather than decide, and local SQLite caching

mitigates, though does not eliminate, dependence on external APIs.

Controlled benchmarking and future work. This report verifies gate correctness under synthetic fault injection, but controlled empirical benchmarking of authoring error rates against unconstrained baselines (formalized in `benchmark_compare.py`) remains future work, as do neural entailment for citation alignment and containerized execution recipes.

Generative AI disclosure. This paper was written using CameraReady, which orchestrates an LLM agent (Claude 3.5/4 Sonnet and Opus, Anthropic) through the ten-phase pipeline described above. The agent was used for literature query formulation, section drafting, and iterative revision under gate supervision. All factual claims, experimental measurements, citation selections, and the decision to submit were reviewed and approved by the human author. Bibliographic entries were resolved exclusively through registry APIs (arXiv, DBLP, CrossRef) as described in Section VII; no reference originates from model parametric memory.

Data and code availability. Open source (MIT) at https://github.com/e96031413/camera-ready (tag `v0.2.0`, commit `29a8fdb6a539f4b4e7cfb86988e6691714146e69`). Reproduction commands and raw outputs: `notes/experiment-results.md`; reference provenance records: `citation-provenance.jsonl`.

X. Conclusion

Citation hallucination, numerical drift, and formatting violations are not exotic failure modes; they are what unconstrained authoring agents do by default. CameraReady's claim is that the mechanically checkable subset of manuscript integrity can be enforced deterministically, by building verification gates into the authoring lifecycle rather than bolting detection on afterward. In practice that means three things: the model's memory never touches the bibliography, phase transitions happen only when the filesystem says so, and venue rules live in schemas rather than prompts. What requires judgment is flagged for the author, what requires scholarship is signed off by the author, and everything in between is checked by scripts that refuse to stay silent.

References

[1] Z. Zhao, Y. Wang, T. Stuart, M. D. Vaan, P. Ginsparg, and Y. Yin, "Llm hallucinations in the wild: Large-scale evidence from non-existent citations," 2026. arXiv:2605.07723.

[2] M. Russinovich, R. S. S. Kumar, and A. Salem, "Phantom references: Hallucinated citations that survive peer review at top-tier conferences," 2026. arXiv:2607.00738.

[3] M. Naser, "How llms cite and why it matters: A cross-model audit of reference fabrication in ai-assisted academic writing and methods to detect phantom citations," 2026. arXiv:2603.03299.

[4] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha, "The ai scientist: Towards fully automated open-ended scientific discovery," 2024. arXiv:2408.06292.

[5] S. Schmidgall, Y. Su, Z. Wang, X. Sun, J. Wu, X. Yu, J. Liu, M. Moor, Z. Liu, and E. Barsoum, "Agent laboratory: Using llm agents as research assistants," 2025. arXiv:2501.04227.

[6] M. Zhao, J. Dong, K. Xu, Z. Hasan, C. Fan, S. Jiang, S. Mao, Y. Ling, L. Zou, T. Zhou, Y. H. Chan, W. Zhang, Z. Zhou, G. Huang, H. Li, W. Cun, Z. Chen, M. Yuan, and Y. Geng, "Scienceflow: A long-horizon agent for ml research, scientific discovery and beyond," 2026. arXiv:2608.14354.

[7] L. Mitchener, A. Yiu, B. Chang, M. Bourdenx, T. Nadolski, A. Sulovari, E. C. Landsness, D. L. Barabasi, S. Narayanan, N. Evans, S. Reddy, M. Foiani, A. Kamal, L. P. Shriver, F. Cao, A. T. Wassie, J. M. Laurent, E. Melville-Green, M. Caldas, A. Bou, K. F. Roberts, S. Zagorac, T. C. Orr, M. E. Orr, K. J. Zwezdaryk, A. E. Ghareeb, L. McCoy, B. Gomes, E. A. Ashley, K. E. Duff, T. Buonassisi, T. Rainforth, R. J. Bateman, M. Skarlinski, S. G. Rodriques, M. M. Hinks, and A. D. White, "Kosmos: An ai scientist for autonomous discovery," 2025. arXiv:2511.02824.

[8] Q. Tang, T. Sun, X. Hu, X. Liu, Y. Chen, Y. Shao, B. Li, C. Lv, C. Xu, C. Huang, C. Li, D. Xue, H. Bai, H. Duan, H. Guo, H. He, H. Chen, H. Shen, J. Yuan, J. Sun, J. Cheng, J. Xu, J. Tong, J. Chen, J. Liu, J. Leng, J. Yu, K. Jiang, K. Xiang, K. Yao, L. Feng, L. Yuan, L. Gao, M. Li, Q. Jia, Q. Sun, S. Ding, S. Gong, S. Zhong, T. J. Zhang, T. Gu, T. Liang, W. Liu, W. Yang, W. Fei, X. Wang, X. Liu, X. Ding, Y. Tang, Y. Wang, Y. Jiang, Y. Yang, Z. He, Z. Chen, Z. Xu, Z. Li, and Z. Huang, "Fars: A fully automated research system deployed at scale," 2026. arXiv:2606.31651.

[9] Y. Du, B. Yu, T. Liu, T. Shen, J. Chen, J. G. Rittig, K. Sun, Y. Zhang, A. Krishnan, Y. Zhang, D. Rosen, R. Pirone, Z. Song, B. Zhou, C. Masschelein, Y. Wang, H. Wang, H. Jia, C. Zhang, H. Zhao, M. Ester, N. Hacohen, T. Head-Gordon, C. P. Gomes, H. Sun, C. Duan, P. Schwaller, and W. Jin, "Accelerating scientific discovery with autonomous goal-evolving agents," 2026. arXiv:2512.21782.

[10] Z. Qian, W. Shi, X. Lin, L. Ling, M. Luo, Z. Wang, Z. Zhang, T. Xu, G. Liu, Z. Zhang, S. Zhang, Z. Wang, Z. Feng, Y. Luo, S. Xu, Y. Chen, Z. Feng, Z. Chen, B. Yuan, B. Wu, H. Wang, and K. Chen, "Story2proposal: A scaffold for structured scientific paper writing," 2026. arXiv:2603.27065.

[11] S. Ren, C. Xie, P. Jian, Z. Ren, C. Leng, and J. Zhang, "Towards scientific intelligence: A survey of llm-based scientific agents," 2026. arXiv:2503.24047.

[12] G. Tie, P. Zhou, and L. Sun, "A survey of ai scientists," 2026. arXiv:2510.23045.

[13] L. Kong, X. Sun, W. Chow, L. Li, K. Q. Lin, X. B. Zhang, S. Wang, R. Li, Q. Wu, W. Gao, Y. Wang, S. Xie, J. Liu, L. Qu, S. Li, L. X. Ng, B. R. Cottereau, Z. Liu, T.-S. Chua, and W. T. Ooi, "Ai for auto-research: Roadmap & user guide," 2026. arXiv:2605.18661.

[14] A. Miyai, M. Toyooka, Z. Zhao, K. Watanabe, T. Yamasaki, and K. Aizawa, "Paper reconstruction evaluation: Evaluating presentation and hallucination in ai-written papers," 2026. arXiv:2604.01128.

[15] D. Rao, E. Wong, and C. Callison-Burch, "Detecting and correcting reference hallucinations in commercial llms and deep research agents," 2026. arXiv:2604.03173.

[16] K. Khajavi, S. Sadeghi, R. Adhikari, and A. Tessier, "Citecheck: Retrieval-grounded detection of llm citation hallucinations in scientific text," 2026. arXiv:2605.27700.

[17] C. Cornelio, T. Ito, R. Cory-Wright, S. Dash, and L. Horesh, "The need for verification in ai-driven scientific discovery," 2025. arXiv:2509.01398.

[18] I. Takahara and T. Mizoguchi, "Toward auditable ai scientists: A hypothesis evolution protocol for llm agents," 2026. arXiv:2607.09195.

[19] Y. Wang, "Firstresearch: Auditable question formation for llm scientific discovery agents," 2026. arXiv:2607.05682.

[20] Z. Wang, H. Li, Z. Yang, Z. Hu, S. Zuo, Y. Zhang, D. Ma, D. Luo, C. Wang, J. Peng, T. Huang, S. Guo, H. Wang, Z. Zhu, S. Han, Y. Cao, B. Chen, X. Chen, K. Yu, and L. Chen, "Externalizing research synthesis and validation in ai scientists through a research harness," 2026. arXiv:2606.18874.

[21] Y. Chen, Y. Quan, X. Lin, and R. Tang, "Where fake citations are made: Tracing field-level hallucination to specific neurons in llms," 2026. arXiv:2604.18880.

[22] M. Dassen, R. Kotula, K. Murray, A. Yates, D. Lawrie, E. Kayi, J. Mayfield, and K. Duh, "Factum: Mechanistic detection of citation hallucination in long-form rag," 2026. arXiv:2601.05866.

[23] D. Abbonato, "Checkifexist: Detecting citation hallucinations in the era of ai-generated content," 2026. arXiv:2602.15871.

[24] M. Li, Z. Lin, and S. Ma, "Source or it didn't happen: A multi-agent framework for citation hallucination detection," 2026. arXiv:2605.08583.

[25] D. Rao and C. Callison-Burch, "Bibtex citation errors in scientific publishing agents: Evaluation and mitigation," 2026. arXiv:2604.03159.

[26] J. Lee, "citecheck: An mcp server for automated bibliographic verification and repair in scholarly manuscripts," 2026. arXiv:2603.17339.

[27] P. Béchard and O. M. Ayala, "Reducing hallucination in structured outputs via retrieval-augmented generation," 2024. arXiv:2404.08189. doi: 10.18653/v1/2024.naacl-industry.19.

[28] Y. Liu, W. Zhang, Z. Yang, Z. Zhang, H. Feng, X. Wang, P. Qiu, Y. Liu, B. Poczos, and J. B. Hong, "Care: Pre-execution command verification for shell-executing llm agents," 2026. arXiv:2607.21642.

[29] B. Yu, C. Jia, J. Chi, X. Liu, Y. Wang, H. Bai, Y. Liu, J. Wei, and J. Zhu, "Tracer: Verifiable generative provenance for multimodal tool-using agents," 2026. arXiv:2605.09934.

[30] Y. She, Y. Liang, and E. Kang, "Safeguarding llm agents from misalignment through provenance analysis," 2026. arXiv:2607.01236. doi: 10.1145/3832783.3837535.

[31] R. Souza, T. Poteet, B. Etz, D. Rosendo, A. Gueroudji, W. Shin, P. Balaprakash, and R. F. da Silva, "Llm agents for interactive workflow provenance: Reference architecture and evaluation methodology," 2025. arXiv:2509.13978. doi: 10.1145/3731599.3767582.

[32] Y. Wu, S. Shen, R. Yang, H. Peng, and B. Hu, "Verify, repair, repeat, or stop? robust stopping for noisy verify-repair loops in llm agents," 2026. arXiv:2607.17641.

[33] T. Sah, V. Srivastava, D. Sah, and K. Jordan, "The verifier tax: Horizon dependent safety success tradeoffs in tool using llm agents," 2026. arXiv:2603.19328.

[34] H. Xu, C. Li, X. Ma, X. Ou, Z. Zhang, T. He, X. Liu, Z. Wang, J. Liang, Z. Chu, R. Liu, R. Mu, D. Tu, M. Liu, and B. Qin, "The evolution of tool use in llm agents: From single-tool call to multi-tool orchestration," 2026. arXiv:2603.22862.

[35] H. Liu, Y. Ming, S. Joty, and C. Zhao, "Harnessing llm agents with skill programs," 2026. arXiv:2605.17734.

[36] R. Ye, X. Pang, J. Chai, J. Chen, Z. Yin, Z. Xiang, X. Dong, J. Shao, and S. Chen, "Are we there yet? revealing the risks of utilizing large language models in scholarly peer review," 2024. arXiv:2412.01708.

[37] A. P. Akella, H. V. Siravuri, and S. Rohatgi, "Pre-review to peer review: Pitfalls of automating reviews using large language models," 2025. arXiv:2512.22145.

[38] L. Zhou, R. Zhang, X. Dai, D. Hershcovich, and H. Li, "Large language models penetration in scholarly writing and peer review," 2025. arXiv:2502.11193.

[39] W. Wu, C. Zhang, Y. Zhao, and T. Bao, "Impact of large language models on peer review opinions from a fine-grained perspective: Evidence from top conference proceedings in ai," 2026. arXiv:2604.19578.

[40] Z. Yin and S. Wang, "Span-level detection of ai-generated scientific text via contrastive learning and structural calibration," 2025. arXiv:2510.00890.

[41] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research (a report from the neurips 2019 reproducibility program)," 2020. arXiv:2003.12206.

[42] E. K. Akdeniz, S. Tekir, and M. N. A. A. Hinnawi, "An end-to-end system for reproducibility assessment of source code repositories via their readmes," 2023. arXiv:2310.09634.

[43] Y.-H. Huang and Y.-S. Lin, "Lumen: Cost-transparent multi-agent pipeline for automated systematic review and meta-analysis," 2026. arXiv:2606.28362.

[44] H.-T. Lin and J.-T. Yeh, "meta-pipe: An llm-agent pipeline for end-to-end automated systematic review and meta-analysis," 2026. arXiv:2606.28363.

[45] X. Chen and X. Zhang, "Large language models streamline automated systematic review: A preliminary study," 2025. arXiv:2502.15702.